



GRUPPENNUMMER: 308

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Übrige Gruppenmitglieder:

Florian Adamczyk

Felix Hollfoth

Felix Neumann (Sprecherin)

Kolja Scriba

Protokoll von:

Björn Becker

bjoern.becker@stud.uni-giessen.de

Matrikel Nr.: 8206290

Modul: Künstliche Intelligenz I
Dozent: Prof. Dr. Christian Heiliger
Semester: Wintersemester 2025/26

INHALTSVERZEICHNIS

	Logbuch	II
1	Fragestellung und Ziel	X
2	Darstellung der Arbeitsprozesse	X
2.1	Vorbereitung	X
2.2	Erste Modelle	X
2.3	Random-Search	X
2.4	Ensemble	XI
2.5	Feature-Engineering	XI
2.6	Meine Erkenntnisse und Ausblick	XI
2.7	Reflexion und Einordnung	XII
3	Bewertung der Gruppenmitglieder	XII
3.1	Kolja	XII
3.2	Felix H.	XII
3.3	Florian	XII
3.4	Felix N.	XII

Logbuch — KI1-Projekt 308

Name: Björn Philipp Becker

Gruppe: 308

Zeitraum: 20.02.2026 – 15.04.2026

Matrikelnummer 8206290

Eintrag 1

Datum: 19.02.2026 & 20.02.2026

Titel: Projektstart — Aufgabe lesen, Repository einrichten

Aus Gruppentreffen: Eigenständig und Gruppentreffen 1 (20.02.2026)

Arbeitsschritte:

- Aufgabenstellung gelesen und verstanden
- Recherche, wie man an den Datensatz kommt (Am Ende aus Übung)
- Über GitLab informiert (im Gruppentreffen dann aber anders entschieden, da keiner sich wirklich damit auskennt, aber Florian ordentlich mit GitHub)
- Wiederholung der Vorlesungsfolien
- Repository geklont
- `nbstripout --install` eingerichtet
- Erste Übersicht über den California Housing Datensatz verschafft

Schwierigkeiten / offene Fragen:

- Wie kann das Projekt funktionieren, wenn wir so verschiedene Zeitfenster haben

Eintrag 2

Datum: 17.03.2026

Titel: Wiederholung der Vorlesung und Einlesen in Ergebnisse der Gruppenmitglieder

Aus Gruppentreffen: Eigenständig

Arbeitsschritte:

- Aufgrund von Tensorflow musste die Python Version von 3.14 auf 3.12 heruntergestuft werden.
- Nachvollziehen des Aufbaus des Git-Workspace. Requirements installiert.
- Lesen und Verstehen der Notebooks vor den Neuronalen Netzwerken, die später zum Ensemble-learning beitragen könnten.
- Nochmals Vorlesung wiederholen und dabei vorläufig Ideen entwickeln, an welche Aufgaben ich mich setzen möchte. (Pooling der geographischen Lage, Transformation der Daten(Feature Engineering), Learning-Rate-Scheduler. Ich weiß aus vorherigen Gruppentreffen, dass Hyperparameter wie Form, Lernrate oder Aktivierungsfunktionen schon ausführlich

behandelt wurden).

zusätzliche Recherche mit Google

- Einlesen in Fortschritt der Gruppenmitglieder (NN-Notebooks)
 - dabei wird klar, dass einige Ideen schon umgesetzt wurden
- Überfliegen von Felix H. Logbuch

Entscheidungen:

- ich möchte mich nicht so sehr mit der Form des NN auseinandersetzen, sondern an genannten Dingen arbeiten
 - Pooling (auch wenn Aussicht auf Erfolg nicht so groß ist)
- ich lege mir eine kleine Analoge Formelsammlung für das Projekt und die Umgebung an.

Schwierigkeiten / offene Fragen:

- Das Ausführen der Notebooks der anderen Gruppenmitglieder dauert sehr lange - sollte ich es im JupiterHub der Uni probieren, welchem ich mehr Rechenleistung adhästiere. (spätere Ergänzung: mir wurde in einem späteren Gruppentreffen verdeutlicht, dass ich mich wohl täuschte)

Eintrag 3

Datum: 18.03.2026

Titel: Anfänge und erste Erkenntnisse

Aus Gruppentreffen: Angelehnt an Gruppentreffen 5 am 17.03.

Arbeitsschritte:

- Aus den Ergebnissen von Kolja und Felix H. haben wir Schlüsse über die Größenordnung von Parametern gezogen. Außerdem war Ergebnis, dass die Aktivierungsfunktion kaum Einfluss auf die Ergebnisse hat, weswegen ich heute nur ReLU nutzte, da diese effizient ist (vgl. [StudySmarter](#) Zugriff: 18.03.).
- Ich wollte mich zuerst mit dem grundsätzlichen Coden eines einfachen Neuronalen Netzes auseinandersetzen. Dazu habe ich Modulimporte und Skalierung der bereinigten Daten aus den Gruppenergebnissen übernommen. Da Felix H. herausfand, dass nur ["MedInc", "AveOccup", "Latitude", "Longitude"] Informationsgewinn bieten und die anderen Daten eher zum Rauschen beitragen, entschied ich mich, nur diese 4 zu benutzen.
- Meine ersten beiden kleinen Netze, die ich mit einer L2 Regularisierung zur Vermeidung von Overfitting (Die Nützlichkeit hatte Kolja beim Gruppentreffen eingebracht), kleinem Dropout und mit EarlyStopping ausgestattet hatte, ergaben ordentliche Ergebnisse mit einem R2 von knapp über 0.7 und wenig Overfitting.
- Wie im Gruppentreffen besprochen, ist die erste Aufgabe, ein sehr gutes Netz aus den bisherigen Erkenntnissen zu bauen. Ich erwartete, da bisher die Hyperparameter eher einzeln getestet wurden, Probleme mit lokalen Minima zu bekommen. Deswegen nutze ich Random-Search. Hyperparametereingrenzung ergaben sich aus den Ergebnissen der Anderen mit Abgleich mit ChatGPT: Ergebnis:
 - Neuronen: 32, 64, 128
 - Layer: 2, 3
 - Learning Rate: $1e-4$ – $1e-2$ (log)
 - L2: $1e-5$ – $1e-3$
 - Dropout: 0.1 – 0.3
 - Batch Size: 16, 32, 64
 - Epochs: max 100 (EarlyStopping)
 - Aktivierung: ReLU

- Ich erwartete eine enorme Rechenzeit und deswegen informierte ich mich über Parallelisierung
- Ergebnis:

	run_id	R ² Val	R ² Test	units	depth	lr	l2	dropout	batch_size	epochs
27	27	0.757183	0.743349	128	2	0.000343	0.000052	0.156997	16	81
33	33	0.751369	0.736966	128	2	0.001588	0.000024	0.163947	16	66
29	29	0.750405	0.736433	128	2	0.000395	0.000017	0.256285	64	93
16	16	0.747135	0.733957	64	3	0.000941	0.000284	0.116963	16	77
13	13	0.746669	0.733977	128	3	0.001095	0.000016	0.195057	32	61

- Ich hoffte Zusammenhänge sehen zu können, aber das ist nicht der Fall. Deswegen werde ich den Code erneut ausführen und die besten Ergebnisse sammeln. Ich erhoffe mir mit einer größeren Menge an Daten eine gescheiterte lineare Regression durchzuführen, um Zusammenhänge zu verstehen.
- Beim nochmaligen darüber Nachdenken bin ich nun nicht mehr so überzeugt, dass die lin. reg so viel bringt. Frage ChatGPT: Welche anderen Möglichkeiten gibt es, die Daten in eine verständliche Form zu bringen? - PDP (Partial Dependence Plots) mit RandomForestGenerator
 - die PDP grenzen die Hyperparameter weiter ein
 - so wie Florian die Feature Importance herausgefunden hat, kann man jetzt die Hyperparameter Importance aus dem Random Forest ausgeben lassen
 - dieses Metamodell zeigt den Zusammenhang zwischen den Hyperparametern und dem R² Score.
 - Jetzt kann man iterativ **Random-**/ oder Grid-Search machen und die Hyperparameter weiter eingrenzen

(siehe dazu in results: pdp.png und Hyperparameter_Importance.png)

Entscheidungen:

- Morgen mache ich das ganze nochmal mit einem größeren Datensatz
- Ich möchte schauen, wie gut die Netze werden, wenn man den Vorgang mehrfach wiederholt

Schwierigkeiten / offene Fragen:

- Macht es Sinn, die Zeit aufzuwenden mehr Daten anzusammeln? Wie gut kann das werden? Laufe ich in eine Sackgasse?

Eintrag 4

Datum: 19.03.2026

Titel: erste Iterationen

Aus Gruppentreffen: 5 vom 17.03.

Arbeitsschritte:

- Ich erweitere meinen Datensatz, um bei den PDP und Hyperparameter-Importances klare Schlüsse ziehen zu können. Bisher besteht mein Datensatz nur aus den 30 besten Einträgen der vorherigen Runs.

- zu sehr werde ich es nicht erweitern, da ich das Gefühl bekomme, Tendenzen jetzt schon abschätzen zu können. Es braucht sehr viel Rechenzeit
 - Es hat sich gelohnt zu warten. Der neue Datensatz mit nun 50 Einträgen zeigt eine andere Tendenz für die Batchsize. (Vorher 32, jetzt 16). Die Tiefe scheint ziemlich egal zu sein. Bei der Hyperparameter-Importance gewinnt die Batchsize stark an Einfluss. Es lässt sich nur mit schwacher Sicherheit sagen, dass die Anzahl der Units pro Layer und die Tiefe eine vergleichbar kleine Rolle spielen. Dagegen sind wahrscheinlich "dropout", "Lernrate" und "L2-Regularisierungsfaktor" von großer Bedeutung.
 - ich finde es etwas Überraschend, dass hier "L2" wichtig ist
 - Das ganze Vorgehen entspricht nicht meiner Intuition. Ich bin mir unsicher, wie die Ergebnisse zu interpretieren sind
 - Ich werde nun trotzdem die Hyperparameter anhand der PDP eingrenzen.
 - Units: bleibt - aber Zufallsverteilung eher Richtung 128
 - depth: das Ergebnis ist ziemlich waagrecht. Ich nehme mal noch die 4 hinzu und schaue was passiert
 - batch size nur noch 16 und 32
 - dropout: Nur noch 0.1 - 0.16
 - lr: bleibt
 - l2: bleibt
 - ich hatte vorher noch die Möglichkeit, dass auch Netze über 50 Epochen gerechnet werden konnten. In den Ergebnissen ist kein Abbruch vor 50 Epochen zu sehen, daher wird es entfernt und auf 100 festgelegt (mit Earlystop).
 - mit den Ergebnissen der anderen Gruppenmitgliedern im Kopf, sind mehr Epochen sehr wahrscheinlich besser, aber die Rechenzeit ist einfach zu enorm, als das ich es ausprobieren könnte. Vielleicht mal über Nacht.
 - Nun habe ich die Ergebnisse:
 - Die neuen Netze neigen maginal weniger zum Overfitting, was auch vorher kein großes problem war. In der Spitze sind die neuen Netze laut dem R2 Wert tatsächlich nicht besser. Der Vergleich ist aber unfair, da jetzt ca. 400 Netze mit 200 Netzen verglichen werden
 - auf Platz 20 für den besten Testscore ist aber eine leichte Verbesserung zu sehen, trotz ungerechtem Vergleich.
 - In der Breite ist es nun besser geworden
 - Aus PDP und Hyperparameter-Importance kann man nun sehen, dass 4 Layer nicht gut sind und eine Einschränkung des Dropouts auf 0.115 - 0.15 Sinn machen könnte. Ich werde nun die Parameter nochmal verändern und das Ergebnis dann zu meinem neuen Datensatz hinzufügen. Außerdem werde ich mal 200 Epochen als Maximalwert ausprobieren
 - 200 Epochen zu nutzen war sehr gut. Die Hyperparameter-Importance zeigt nun, dass der wichtigste Parameter nun die Lernrate ist.
-  Hyperparameter-Importance
-  PDP Nr.3
- Mit Blick auf die Lernrate stellt sich die Frage, ob das erlaubte Intervall optimal angesetzt ist. Speziell werde ich mir kleinere Lernraten ansehen ab 10^{-6} .
- Die Ergebnisse werden nicht besser

Entscheidungen:

- Ensemble aus Netzen mit den gesamten California Housing testen. Ich habe noch Baysche Netze im Kopf(ohne genau zu wissen was das ist, aber das passt glaube ich zu dem Thema.
Das ist eher eine Rechercheaufgabe)

Schwierigkeiten / offene Fragen:

- Der heutige Tag war vor allem von warten geprägt. Die Ausführung des Codes dauert jeweils mehr als eine Stunde.

Eintrag 5

Datum: 20.03.

Titel: erstes Ensemble

Aus Gruppentreffen: 5 vom 17.03.

Arbeitsschritte:

- Jetzt ein letztes mal Parameteranpassung. Lernrate (logarithmisch) wird auf Intervall $[-3.3; -2]$ begrenzt.
- Über Nacht habe ich darüber nachgedacht, ob es eine gute Entscheidung war, immer wieder die schlechten Netze zu löschen. Vielleicht könnten diese Ergebnisse das Gesamtergebnis verbessern.
- Die Verbesserung hat gute Ergebnisse erzielt. Ich werde die Epochen auf 500 hochschrauben, da viele Netze die 200 max Epochen zum lernen nutzen. Diese Netze sind wirklich gut, aber kaum entschieden besser als vorher.
- Ich hatte vorher schon die Units gewichtet. Vorher war es stark favorisiert in Richtung 128, jetzt immer noch, aber weniger stark. Ich möchte mehr diversifizieren, da ich nur NN für die späteren Ensembles benutze, was sehr wichtig ist (vgl. Vorlesung 7, S.11). Ich nehme eine Hand voll der Netze und benötige noch ein paar schwächere mit stark anderen Hyperparametern
- ich dachte eigentlich, dass wir einen gemeinsamen Speicher von unseren besten Netzen haben, aber das einzige, dass ich fand, war eine leere Liste, die nur Einträge im Arbeitsspeicher vermerkt.
- Ich habe jetzt ein Ensemble gebaut. Leider ist das Ergebnis mit $R^2 = 0.7499$ nur wenig besser als der Durchschnitt der einzelnen Netze.
-

Entscheidungen:

- Aufgrund der nicht so bereichernden Ergebnisse heute, möchte ich morgen das gleiche mit allen Features machen.
 - entweder der Score wird besser, oder der Score wird schlechter. Im zweiten Fall müsste ich dann die Hyperparameter nochmal neu anpassen. - Also wieder Random-Search
- Außerdem finde ich spannend die Features zu verknüpfen

Schwierigkeiten / offene Fragen:

- Es ist schwer die Ergebnisse einzuordnen, da nicht hinreichend unterschieden werden kann zwischen statistischen Schwankungen und Wahrheit.

Eintrag 6

Datum: 21.03.

Titel: Feature Anpassung

Aus Gruppentreffen: Eigenständig, aber aufbauend auf Gruppentreffen 5

Arbeitsschritte:

- Ich trainiere das Ensemble nun auf den gesamten California Housing Datensatz. Der Score ist 0.78 und damit deutlich besser als zuvor. Ich werde das Ensemble noch ein paar mal ausführen, um abzuschätzen, wie groß das Rauschen ist. (Ergebnisse: 0.778, 0.7795, 0.788, 0.7807, 0.7809)
- Fragte ChatGPT: welche Features des California Housing Datensatz kann man verbinden, sodass ein besseres NN entsteht?
 - ... Diese sind besonders effektiv: AveRooms / AveOccup, AveBedrms / AveRooms, Population / AveOccup, MedInc /

AveOccup ... Es sollten die normierten Daten verwendet werden.

- Ich habe heute keine Lust mehr mich mit Python Fehlern auseinander zu setzen

Eintrag 7

Datum: 23.03.

Titel: Anfang Dokumentation

Aus Gruppentreffen: Eigenständig

Arbeitsschritte:

- Habe einen schwer zu identifizierenden Fehler. Der Score des aktuellen Ensembles ist ?negativ?.
 - in meinen Daten ist ein einziger Wert nicht Non-Null. Grund könnte ein geteilt durch 0 Problem beim Feature Engineering sein, dass ich durch das Hinzufügen eines kleinen Wertes in den Nenner zu beheben versuche.
 - Der score ist nun 0.65
 - Außerdem ist die Skalierung im Eimer. Wahrscheinlich macht es Sinn, erst Feature Engineering mit den Ursprungsdaten zu machen und dann alle gemeinsam zu skalieren.
 - Der Score von nun 0.765 ist etwas enttäuschend
- Nach einer längeren Lagebesprechung mit Felix N. haben wir uns für heute dazu entschieden, wie auch im Gruppentreffen 5 vereinbart, schonmal die Ergebnisse des Projektes zusammen zu fassen. Gesprächsthemen waren außerdem der aktuelle Stand und formale Themen. Außerdem werde ich mich im Verlauf des heutigen Tages mit meinem Notebook beschäftigen in Sachen Lesbarkeit und Dokumentation.

Eintrag 8

Datum: 24.03

Titel: Enttäuschende Ergebnisse

Aus Gruppentreffen: 5

Arbeitsschritte:

- Ich habe nun endlich den Fehler in meinem Code gefunden und noch einen weiteren aufgedeckt. Er betrifft das gestrige Ensemble(vor feature Engineering). Ich hoffe nun bessere Ergebnisse zu bekommen. - 0.768
- Ich habe nun also noch einmal Random-serach verwendet, um für den erweiterten Datensatz gute netze zu finden. Allerdings ist das Ergebnis mit 0.67 wirklich schwach. ich muss wahrscheinlich nochmal die Parameterspannen erweitern.
 - möglich wäre auch dass die maximale Anzahl von 200 Epochen zu wenig ist, um die Netze hinreichend zu trainieren. Deswegen nun in allen Hyperparametern größere Spannen und mehr Epochen. - Das hat wieder gar nichts gebracht.
- Die nächste Idee war, die Architektur ein bisschen zu diversifizieren. Das bringt ein bisschen Verbesserung, aber nicht relevant.
- Ich speichere das trotzdem mal ab. Ganz zum Schluss möchte ich ein weighted Ensemble bauen. Dafür brauche ich die Bestandteile so verschieden wie möglich.
 - ich muss dann noch den Shrink-Faktor zwei bei den anderen gespeicherten Netzen hinzufügen.
- was ich beim feature-Engineering noch ausgelassen habe, ist y zu verändern. Ich erhoffe mir wenig davon, aber ein Versuch ist es Wert. Man könnte sich da viel überlegen, aber ich nehme der Einfachheit halber die Ursprungsdaten, also

nicht wie sonst die Normalisierten. Das spuckt Fehler aus, die ich nicht identifizieren kann. Ich werde mich bis ich wieder motivation zur Fehlersuche gesammelt habe etwas anderes.

- Ich bin mittlerweile zu dem Schluss gekommen, dass die neu gewählten Features einfach schlecht waren.
 - Ich habe dazu ChatGPT gefragt, was bessere Features sind: "kannst du mir bessere features zum engineeren vorschlagen"
 - "MedInc" * "AveRooms"
 - "MedInc" * "Population"
 - Distanz zum Zentrum: $(\text{"Latitude"} - 36)^2 + (\text{"Longitude"} + 119)^2(-1/2)$
 - "AveOcc"²
 - $\log(\text{"Population"})$
 - ich ersetze also...
- nach 4 Stunden Ladezeit ist auch hier der score unter 0.7, was wieder nicht so optimal ist.

Eintrag 9

Datum: 25.03.

Titel: Letzes Ensemble

Aus Gruppentreffen: Eigenständig

Arbeitsschritte:

- Ich nutze nochmal Random-Search und füge einen neuen Hyperparameter hinzu. Ich möchte, dass die Architektur nicht jedes mal so aussieht, dass die Layer sich in der Größe halbieren.
- Habe ein neues Ensemble gebaut mit dieser Erweiterung und mit den drei verschiedenen Datensätzen getestet
- Beginne mit dem Schreiben meiner Logbuchzusammenfassung

Eintrag 10

Datum: 26.03.

Titel: Dokumentation

Arbeitsschritte:

- Aufgrund von Problemen mit Git sind meine Zellenoutputs nicht mehr einsehbar.
 - Leider habe ich die Ergebnisse von den Ensembles nicht dokumentiert.
- Schreibe heute an meiner Zusammenfassung weiter
- Heute Abend ist unser letztes Gruppentreffen

Eintrag 11

Datum: 26.03.

Titel: Dokumentation

Arbeitsschritte:

- Ich finde leider nicht mal mehr den Code (siehe Eintrag 9 und 10) in unserem Git und auch nicht im Git-Verlauf. Ich bin mir sicher, gespeichert zu haben...
 - da ich eigentlich schon über den 75 Stunden bin und noch Dokumentation und Bewertung der Gruppenmitglieder schreiben muss, werde ich das jetzt auf sich beruhen lassen. Ich weiß noch, dass der Score mein bestes bisheriges Ensemble nicht übertraf, was mich zumindest etwas beruhigt. Das Ensemble ist auch nicht mehr zu finden.
- Ich schreibe an meiner Dokumentation und überarbeite mein Notebook ein kleines bisschen.
- Ich schreibe die Bewertung der Gruppenmitglieder.

Eintrag 12

Datum 27.03. und 30.03.

Titel Korrekturlesen und Abgabe fertig machen

Arbeitsschritte

- An beiden Tagen schaue ich über das Logbuch und schreibe die Gruppentreffen in eine Datei. Markdowns werden zu PDF und das Notebook ohne nbstipout ausgeführt.

Neuronales Netz für den kalifornischen Hauspreis-Datensatz

Björn Becker^a^aJustus-Liebig-Universität Gießen, Matrikelnummer: 8206290

Gruppe 308 – KI I Projekt WS 2025/26

Zusammenfassung—Dieses Dokument fasst meinen Arbeitsfortschritt bei der Entwicklung Neuronaler Netze für die Vorhersage von Hauspreisen anhand des "California housing dataset" zusammen. Es werden vor allem die Hyperparameteroptimierung und Ensemble Erstellung behandelt. Das beste Modell erreicht einen R^2 -Wert von ... auf dem Testset und übertrifft damit die lineare Regression ($R^2 = 0,63$) deutlich.

1. FRAGESTELLUNG UND ZIEL

Ziel des Projekts war es, ein Neuronales Netz aufzubauen, welches die Hauspreise im California housing dataset vorhersagen kann. Neuronale Netze stellen eine zentrale Methode der modernen Künstlichen Intelligenz dar (Russell & Norvig, 2021). Das Ergebnis sollte mit einer linearen Regression verglichen werden, um den Mehrwert komplexerer Modelle zu bewerten. Aufgrund der zeitlichen Verfügbarkeit der Gruppenmitglieder hatte ich die Chance die Gruppenergebnisse zusammenzuführen und weiter zu verwenden. Daher lag der Fokus meiner Arbeit auf der Hyperparameteroptimierung durch die Random-Search-Methode und der Erstellung von Ensembles.

2. DARSTELLUNG DER ARBEITSPROZESSE

2.1. Vorbereitung

Vor unserem ersten Gruppentreffen habe ich mich mit der Aufgabe vertraut gemacht und bin die Vorlesungsfolien durchgegangen. Zusätzlich informierte ich mich über die Einrichtung einer gemeinsamen Programmierumgebung über GitLab. Allerdings kam im Verlauf des ersten Gruppentreffens auf, dass insgesamt in der Gruppe mehr Erfahrungen mit GitHub vorhanden waren, weshalb wir uns dafür entschieden. Florian übernahm die Einrichtung des Repositories und die Architektur der Ordnerstruktur (vgl. Logbuch von Florian). Zwischen dem ersten und 5. Gruppentreffen lag mehr als ein Monat, in dem ich lediglich an den Treffen teilnahm, Florian, Felix H. und Kolja bereits den wesentlichen Teil ihres Arbeitspensums erledigten. Nachdem ich mich einen Tag vor dem fünften Treffen mit dem Fortschritt der Gruppe und den im Repository zugänglichen Dateien, u.a. den Hilfsfunktionen für die Modellevaluation vertraut machte, machte ich mir anhand der Vorlesungsfolien Gedanken über mögliche Aufgaben, an denen ich arbeiten könnte. Außerdem habe ich via venv eine passende Python-Umgebung eingerichtet und die benötigten Pakete installiert, um die Jupyter-Notebooks der Gruppe ausführen zu können.

2.2. Erste Modelle

Nach dem Gruppentreffen, habe ich zuerst zwei eigene Neuronale Netze gebaut, um die Anwendung der Keras-Bibliothek zu erlernen und den Gebrauch einiger Hilfsfunktionen zu verstehen. Auf Grundlage der Gruppenergebnisse entschied ich mich bei allen weiteren Netzen für die Verwendung der Aktivierungsfunktion `relu` und des Optimierers `Adam` (vgl. Logbuch von Felix H.). Ich orientierte mich auch bei Hyperparameterwahl und Struktur des Netzes, d.h. kleiner werdenden Schichten, an den bereits erstellten Modellen der Gruppe. Zudem wurde aus den Gruppenergebnissen deutlich, dass nicht alle Features für die Vorhersage relevant sind, weshalb ich mich im Folgenden nur mit den Features `MMedInc`, `AveOccup`, `LLatitude` und `LLongitude` beschäftigte. Die Auswahl relevanter Features ist ein zentraler Bestandteil von Data-Mining-Prozessen (Runkler, 2012), was die Vorgehensweise bestätigt. Mit den Netzen erreichte ich R^2 -Werte

von 0.7188 und 0.7193, was den Vergleich mit der linearen Regression mit $R^2 = 0.7091$ (vgl. `01_Lasso_Ridge.ipynb`) bereits übertraf.

2.3. Random-Search

Nun beginnt der Hauptteil meiner Arbeit. Für alle Hyperparameter war von den Gruppenmitgliedern eine ungefähre optimale Größenordnung vorgegeben worden. Diese habe ich verwendet, um gute Modellkonfigurationen zu generieren. Problematisch erschien mir, dass bisher die Hyperparameter jeweils einzeln variiert wurden, was dazu führen kann, dass bei einer geordneten Suche nicht die optimale Kombination gefunden wird, da der Algorithmus eher Plateaus findet als optimale Kombinationen. Dies ist darauf zurückzuführen, dass Hyperparameter häufig Wechselwirkungen aufweisen und daher nicht unabhängig voneinander betrachtet werden sollten (Ertel, 2021). Daher entschied ich mich für die Random-Search-Methode, bei der die Hyperparameter zufällig aus den vorgegebenen Bereichen ausgewählt werden. Zu Beginn wurden diese gewählt, wie in Tabelle 1 dargestellt. Random Search gilt als effektive Methode zur Hyperparameteroptimierung, da sie im Vergleich zu systematischen Suchverfahren häufig schneller gute Kombinationen findet (vgl. Bergstra & Bengio, 2012).

Hyperparameter	Wertebereich
Anzahl Schichten	2, 3
Anzahl Neuronen pro Schicht	32, 64, 128
Lernrate	0.0001 bis 0.01
Dropout-Rate	0.1 bis 0.3
Batchgröße	16, 32, 64
Anzahl Epochen	50, 100
L2-Regularisierung	0.00001 bis 0.001
Aktivierungsfunktion	ReLU
Optimierer	Adam
EarlyStopping	Ja

Tabelle 1. Hyperparameter und Wertebereiche für die Random-Search

Damit die Rechenzeit im Rahmen blieb nutze ich Parallelisierung, womit mehrere Modelle gleichzeitig trainiert werden konnten. Aus mehreren hundert Modellen, wählte ich die besten 50 aus, um diese zu analysieren.

R^2 -Valuation	R^2 -Test	units	depth	learning_rate	l2_reg	dropout_rate	batch_size	epochs
0.757183	0.743349	128	2	0.000343	0.000052	0.156997	16	81
0.751369	0.736966	128	2	0.001588	0.000024	0.163947	16	66
0.750405	0.736433	128	2	0.000395	0.000017	0.256285	64	93
0.747135	0.733957	64	3	0.000941	0.000284	0.116963	16	77
0.746669	0.733977	128	3	0.001095	0.000016	0.195057	32	61

Tabelle 2. Ergebnisse der Random-Search: Hyperparameterkonfiguration einiger der besten Modelle.

Aus den Ergebnissen waren keine klaren Aussagen mit bloßem Auge möglich, weshalb ich mich dazu entschied einen Random-Forest-Classifer mit den Hyperparametern zu trainieren und damit die Hyperparameter-Importance (Abb. 1) bzw. den Einfluss der Hyperparameter auf die Modellleistung zu erzeugen. Außerdem ließ ich mir die im sklearn-Modul verfügbaren Partial-Dependence-Plots anzeigen, um die Auswirkungen der Hyperparameter auf die Modellleistung zu verstehen.

Mit den Partial-Dependence-Plots (Abb. 2) war es dann möglich, die Auswirkungen der Hyperparameter auf die Modellleistung zu verstehen. Durch ein Iteratives Vorgehen, bei welchem nach jeder Iteration, d.h. nach der Erstellung von einigen hundert Modellen, die Hyperparameter-Importance analysiert und die Wertebereiche der Hyperparameter angepasst wurden, konnte das Modell sukzessive

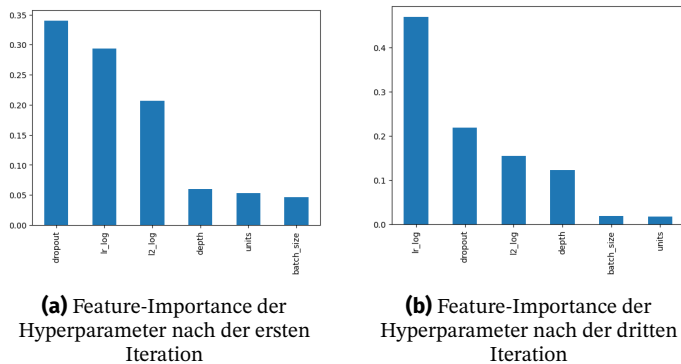
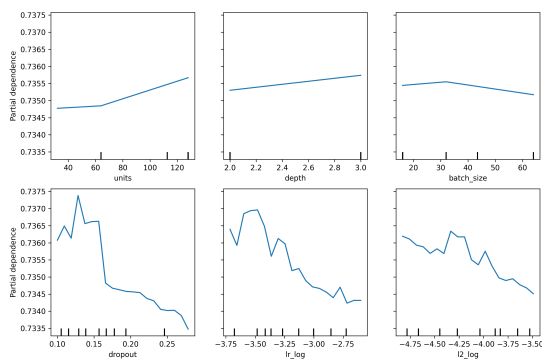
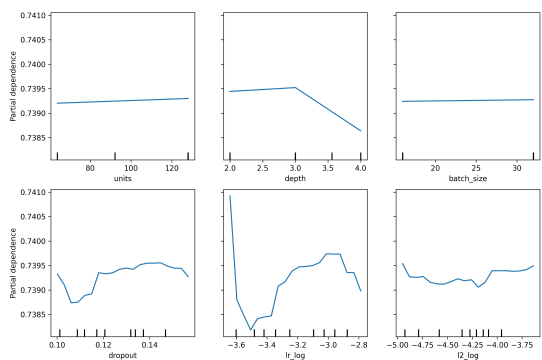


Abbildung 1. Vergleich der Hyperparameter-Importance nach der ersten und dritten Iteration der Random-Search.

verbessert werden. Allerdings zeigte sich nach der zweiten Iteration schon keine große Veränderung mehr, weshalb ich das Vorgehen nach der dritten Iteration beendete.



(a) Partial-Dependence-Plots nach der ersten Iteration



(b) Partial-Dependence-Plots nach der dritten Iteration

Abbildung 2. Partial-Dependence-Plots nach der ersten und dritten Random-Search-Iteration.

Währenddessen probierte ich auch größere Spannen bei den Hyperparametern aus, um zu sehen, ob es vielleicht doch noch bessere Modelle gibt. Zu nennen ist hier zum Beispiel die Tiefe des Netzes oder eine kleinere Lernrate. Ziel war es, etwas über den Tellerrand hinauszuschauen, um zu prüfen, ob es vielleicht doch noch bessere Modelle gibt. Es zeigte sich, dass die Tiefe des Netzes von 4 Layern gute Ergebnisse lieferte, weshalb ich diesen Wert in die Random-Search mit aufnahm. Vorerst nutze ich eine maximale Epochenzahl von 100. Da trotz Early-Stopping aber nach der zweiten Iteration die Netze noch nicht ausreichend trainiert worden waren, erhöhte ich die

maximale Epochenzahl. Schlussendlich passte ich die Hyperparameterbereiche folgendermaßen an (Tab. 3):

Hyperparameter	Wertebereich
Anzahl Schichten	2, 3, 4
Anzahl Neuronen pro Schicht	32, 64, 128
Lernrate	$10^{-3.3}$ bis 10^{-2}
Dropout-Rate	0.115 bis 0.16
Batchgröße	16, 32, 64
Anzahl Epochen	500
L2-Regularisierung	0.00001 bis 0.001
Aktivierungsfunktion	ReLU
Optimierer	Adam
EarlyStopping	Ja

Tabelle 3. Hyperparameter und Wertebereiche für die Random-Search

Das beste gefundene Netz erreichte einen R^2 -Wert von 0.752 auf dem Testset, was eine deutliche Verbesserung gegenüber der linearen Regression darstellt.

2.4. Ensemble

Nachdem ich 10 meiner besten Hyperparameterkonfigurationen aus der Random-Search-Phase ausgewählt hatte, entschied ich mich, ein Ensemble zu erstellen. Ensemble-Methoden werden eingesetzt, um durch die Kombination mehrerer Modelle die Generalisierungsfähigkeit zu verbessern (Kruse et al., 2013). Dieses Ensemble verbesserte den Score nicht, sondern erreichte lediglich einen R^2 -Wert von 0.7499, was trotzdem besser war als die allermeisten Einzelmodelle. Um eine Aussage über den gesamten California-Housing-Datensatz zu treffen, trainierte ich das Ensemble auf eben diesen und erreichte einen R^2 -Wert von 0.788, was eine deutliche Verbesserung gegenüber dem verkleinerten Datensatz darstellte.

2.5. Feature-Engineering

Aus dem deutlichen Sprung des R^2 durch die Hinzunahme der restlichen Features lag nahe, Feature-Engineering zu betreiben. Tatsächlich konnte ich durch die Erstellung neuer Features (siehe Tab. 4 u. Tab. ??) das Ensemble in zwei Versuchen nicht verbessern, sondern verschlechterte es sogar.

neues Feature	Definition
rooms per household	AveRooms/AveOccupancy
bedrooms per room	AveBedrms/AveRooms
population per household	Population/AveOccupancy
income per household	MedInc/AveOccupancy

R^2 -Wert 0.65

Tabelle 4. zusätzliche Features und deren Definitionen im ersten Versuch des Feature-Engineerings

neues Feature	Definition
income · rooms	MedInc × AveRooms
income · population	MedInc × Population
distance to center	$\sqrt{(Latitude - 36)^2 + (Longitude + 119)^2}$
squared Occupancy	AveOccup × AveOccup
logarithm of population	$\log(Population)$

R^2 -Wert 0.68

Tabelle 5. zusätzliche Features und deren Definitionen im zweiten Versuch des Feature-Engineerings

2.6. Meine Erkenntnisse und Ausblick

Kern meiner Arbeit war die Hyperparameteroptimierung durch die Random-Search-Methode und Ensembles auf verschiedenen Datensätzen zu trainieren. In meiner Arbeit habe ich erkannt, dass es sehr schwierig ist, Neuronale Netze anhand von Hyperparameteroptimierung zu verbessern. Oft kommen Verbesserungen überraschend, wie

bei der Hinzunahme der als unwichtig deklarierten Features und manchmal haben Entscheidungen die logisch erscheinen, wie das daraus gefolgte Feature-Engineering, negative Auswirkungen auf die Güte des Netzes. Während meiner Arbeit habe ich eher gelernt, wie man sicher viele gute Netze generiert und weniger wie man ein sehr gutes erhält. Es konnte in meiner Arbeit gezeigt werden, dass Neuronale Netze vor allem mit Hyperparameteroptimierung durch Random-Search-Methoden die Zusammenhänge innerhalb des California-Housing-Datensatzes wesentlich besser erfassen können als lineare Regressionen. Weitere Verbesserungen könnten z.B. durch das Untersuchen der Architektur der Netze, mit Hilfe eines Verkleinerungsfaktors für die Schichten, erreicht werden.

2.7. Reflexion und Einordnung

Insgesamt konnte mein bestes Modell einen R^2 -Wert von 0.788 auf dem gesamten Datensatz erreichen, was für wirtschaftliche Anwendungen bereits als gutes Ergebnis interpretiert werden kann (Investopedia, o.J.), wobei diese Interpretation stark kontextabhängig ist. Online findet sich eine Ausarbeitung des Datensatzes, bei der ein Neuronales Netz mit einem R^2 -Wert von etwa 0.80 auf dem Testset erreicht wurde (medium, o.J.), was zeigt, dass das erreichte Ergebnis konkurrenzfähig ist, aber auch noch Raum für Verbesserungen bietet. Die Methodik der Hyperparameteroptimierung durch Random-Search-Methoden hat das Problem, dass es keine Garantie dafür gibt, dass die optimale Kombination gefunden wird. Es muss bei jedem Hyperparameter in einem kleinen Intervall gesucht werden, wobei nicht ausgeschlossen werden kann, dass die optimale Kombination außerhalb dieses Intervalls liegt. Die gebildeten Ensembles haben die Vorhersageleistung kaum verbessert. Als nächster Schritt sollte in das Ensemble andere Modelle neben Neuronalen Netzen aufgenommen werden, um die Diversität der Modelle zu erhöhen. Außerdem ist hat auch das Rauschen der Modelle einen Einfluss auf die Güte des Ensembles. Das Feature-Engineering hat sich als schwierig erwiesen und sich negativ auf die Güte des Netzes ausgewirkt. Möglicherweise könnten aber andere Features auch einen positiven Einfluss haben, weshalb es sich lohnen könnte, weitere Features zu erstellen und zu testen. Es sei noch erwähnt, dass der Datensplit in Trainings-, Validierungs- und Testset immer der gleiche war, um bessere Vergleichbarkeit zu gewähren. Allerdings sind die erzielten Ergebnisse nicht unbedingt auf alle anderen möglichen Splits übertragbar.

3. BEWERTUNG DER GRUPPENMITGLIEDER

Bevor ich die Bewertung der Gruppenmitglieder vornehme, möchte ich kurz darauf hinweisen, dass aufgrund der zeitlichen Planung, ich nur in den Gruppentreffen und ein kleines bisschen darüber hinaus mit Kolja, Felix H. und Florian in Kontakt stand. Daher ist die Bewertung hier etwas oberflächlich.

3.1. Kolja

Kolja hatte eine Phase im ersten Gruppentreffen, in der er mit seinen Problemen bezüglich Einrichtung seiner Programmierumgebung beschäftigt war, was dazu führte, dass er zwischenzeitlich etwas abwesend wirkte. Allerdings wurde in dieser Phase etwas weniger Entscheidendes besprochen, weshalb es nicht zu einem großen Problem wurde. Andererseits erlebte ich Kolja als einen sehr warmen und freundlichen Menschen. Ich erinnere mich daran, dass er sich nach einem Gruppentreffen Zeit nahm, um mit mir über Schwierigkeiten im Studium und bei der Wohnungssuche zu sprechen. Kolja ist definitiv der Typ Mensch, auf den man immer zugehen kann. Er leitete am Ende des letzten Gruppentreffens eine mediatorische Diskussion ein, um eventuelle Meinungsverschiedenheiten zu klären, was bei allen gut ankam, ohne dass etwas zu besprechen war.

3.2. Felix H.

Felix H. war sehr engagiert dabei und schaffte mit seinen Erkenntnissen die Grundlage für meine Arbeit. Unsere Gruppentreffen verliefen trotz vorhandener Agenda etwas chaotisch ab, wobei Felix sich immer um Struktur in diesen und im Rest des Projektes bemühte. Leider konnte er sich nicht immer durchsetzen. Man konnte Felix ansehen, wie er im Laufe des Projektes mehr und mehr vertraut mit der gesamten Materie wurde. Ich behaupte, dass er sich am Ende des Projektes am besten mit Neuralen Netzen auskennt.

3.3. Florian

Florian war unsere Bank und erster Ansprechpartner bei allen technischen Problemen. Hochmotiviert stürmte er sich auf die Einrichtung unseres Workspaces. Manchmal wirkte es allerdings, als sei er in seiner eigenen Welt. Damit meine ich, dass er in Gruppentreffen manchmal vom Thema abschweifte, um seine Gedanken zu teilen, die nicht unbedingt zum Thema passten. Dass er auch am Ende unsicher mit den Namen der Gruppenmitglieder war, bestätigt diesen Eindruck. Andererseits war er immer sehr freundlich und hilfsbereit. Er brachte einen nicht zu bändigen Elan mit, welcher auf mich ansteckend wirkte. Seine Arbeit ermöglichte es uns, uns mit der eigentlichen Materie zu beschäftigen. Der Workspace ist sehr gut organisiert und seine Notebooks sehr gut dokumentiert, was mir sicherlich einige Zeit und Frustration beim Einlesen zu Beginn meiner Arbeit gespart hat.

3.4. Felix N.

Da wir im selben Studiengang und Semester sind, hatten wir schon vor dem Projekt einige Berührungspunkte. Felix N. ist ein klassischer Menschenfänger. Seine Art mit Menschen umzugehen und Menschen zu vernetzen ist wahrlich bewundernswert. Felix und ich standen während unserer Bearbeitung immer mal wieder in Kontakt, wobei wir uns über unseren Arbeitsfortschritt und das weitere Vorgehen austauschten. Seine Motivation für das Projekt war nicht besonders hoch, was er mir gegenüber äußerte, aber für mich nicht erkennbar in seiner Arbeitsweise war. Die Zuverlässigkeit, die er in seinen Aufgaben als Gruppensprecher an den Tag legte, sorgte auch dafür, dass ich mehr Zeit hatte um verschiedene Dinge auszuprobieren. Obwohl er am kürzesten Zeit hatte, um am Neuronalen Netz zu arbeiten, konnte sein Ergebnis, dieses aller Gruppenmitglieder übertreffen.

LITERATUR

- [1] Felix Hollfoth. *Logbuch – KI I Projekt, Gruppe 308*. Unveröffentlichtes Arbeitsdokument, 2026.
- [2] Florian Adamczyk. *Logbuch – KI I Projekt, Gruppe 308*. Unveröffentlichtes Arbeitsdokument, 2026.
- [3] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. 4th Edition, Pearson, 2021.
- [4] Wolfgang Ertel. *Introduction to Artificial Intelligence*. 3rd Edition, Springer, 2021.
- [5] Thomas Runkler. *Data Mining: Methoden und Algorithmen intelligenter Datenanalyse*. Vieweg+Teubner, 2012.
- [6] Rudolf Kruse, Christian Borgelt, Frank Klawonn, Christian Mowes, Matthias Steinbrecher. *Computational Intelligence: Eine methodische Einführung in Künstliche Neuronale Netze, Evolutionäre Algorithmen und Fuzzy-Systeme*. Springer, 2013.
- [7] James Bergstra and Yoshua Bengio. Random Search for Hyperparameter Optimization. *Journal of Machine Learning Research*, 13:281–305, 2012.

- [8] Investopedia. *R-Squared*. Verfügbar unter: <https://www.investopedia.com/terms/r/r-squared.asp> (Zugriff am 26.03.2026).
- [9] Advika *California Housing Price Prediction: A Multi-Model Analysis* Verfügbar unter: <https://medium.com/@saudhaminiupdated/california-housing-price-prediction-a-multi-model-analysis-e63bd9cbddb1> (Zugriff am 27.03.2026)